

Data Mining Project Report

Title

Predicting Monetary Relief in CFPB Complaints

Names

Amin Ben Abdelhafidh

Koussay Hidouri

Louay Ilahi

Course Title

BA 360: Business Data Mining

GitHub Repository

<https://github.com/aminbadh/cfpb-complaint>

Contents

1	Dataset Description	1
1.1	General Context	1
1.2	Data Source	1
1.3	Unit of Observation	1
1.4	Main Variables	1
2	Problem Statement	2
2.1	Problem to Study	2
2.2	Objective	2
2.3	Research Questions	2
2.4	Target Variable	2
3	Data Preparation	3
3.1	Preparation Workflow Across Notebook 1 and Notebook 2	3
3.2	Stage 1: EDA-Driven Preparation Decisions (Notebook 1)	3
3.3	Stage 2: Authoritative Preprocessing (Notebook 2)	3
3.3.1	Target Construction and Eligibility Filter	3
3.3.2	Text Cleaning and Narrative Features	4
3.3.3	Temporal Feature Engineering	4
3.3.4	Categorical Encoding and Matrix Assembly	5
3.3.5	Missing-Value Handling	5
3.3.6	Leakage Control	6
3.4	Exported Artifacts for Modeling	6
4	Methods and Analysis	6
4.1	Modeling Strategy	6
4.2	Models Implemented and Rationale	7
4.3	Evaluation Protocol	7
4.4	Methods Outputs	7
5	Results and Interpretation	8
5.1	Default vs Tuned Performance	8
5.2	Curve-Level Interpretation	8
5.3	Confusion-Matrix Insights	9
5.4	Key Findings	9
6	Conclusion	10
6.1	Objective Recap	10
6.2	Methods Recap	10
6.3	Main Results	10
6.4	Business Insights	10
6.5	AI Tool Disclosure	11

1 Dataset Description

1.1 General Context

The dataset comes from the U.S. Consumer Financial Protection Bureau (CFPB). It contains real consumer complaints about financial products and services (for example credit cards, loans, credit reporting, and debt collection). In business terms, this dataset helps identify where customer harm occurs and where monetary relief is more likely, which supports better risk monitoring and faster case prioritization.

1.2 Data Source

- Source: CFPB Consumer Complaint Database.
- Official page: <https://www.consumerfinance.gov/data-research/consumer-complaints/>
- Full raw input can be downloaded via the project helper and stored under `data/raw/` when needed for large-scale runs.
- Because the full file is very large, we created and used a memory-safe working sample for modeling: `data/processed/complaints_sample.csv`.

Local snapshot (as documented in the project README):

- Full raw dataset size: 14,636,145 rows and 18 columns.
- Uncompressed CSV size: about 8.0 GB.
- Compressed download size (`complaints.csv.zip`): about 1.7 GB.

1.3 Unit of Observation

One observation (one row) represents one consumer complaint submitted to the CFPB complaint system. Each row includes complaint metadata available at intake time (for example product, issue, submission channel, state, and dates), and in many cases a free-text complaint narrative.

For this project, we defined a binary target from resolution information:

- 1: complaint ended with monetary relief.
- 0: complaint ended without monetary relief.

To avoid leakage, post-resolution fields are not used as predictors at inference time.

1.4 Main Variables

Main variables used in EDA and modeling include:

- Text variable: `consumer_complaint_narrative`.
- Product taxonomy: `product` and `sub_product`.
- Problem taxonomy: `issue` and `sub_issue`.
- Submission and geography: `submitted_via` and `state`.
- Time variables: `date_received` (and derived temporal features).
- Target construction fields: `company_response_to_consumer` (used to derive the label, then excluded from predictor set for leakage-safe modeling).

Data handling note: because the full raw dataset is too large for efficient iterative experimentation

on a local machine, we generated a representative sample (data/processed/complaints_sample.csv) and used that sample throughout preprocessing and model development.

2 Problem Statement

2.1 Problem to Study

Financial institutions and regulators receive a very large volume of consumer complaints, but only a subset of cases ends with monetary relief for the consumer. The core problem is to identify, as early as possible, which complaints are more likely to lead to monetary relief using only information available at complaint time.

This is a pressing business problem for companies because slow or inaccurate complaint triage increases regulatory exposure, operational costs, customer dissatisfaction, and reputational risk. In practice, firms must quickly decide which complaints need urgent escalation versus standard handling. A reliable predictive signal can improve response prioritization, shorten time-to-resolution for high-risk cases, and support more consistent consumer-outcome management.

2.2 Objective

The objective of this analysis is to build and evaluate a leakage-safe binary classification model that predicts whether a CFPB complaint will end with monetary relief. We aim to compare multiple modeling approaches, assess class-imbalance-aware performance metrics, and identify a practical decision threshold for operational use.

2.3 Research Questions

The main research questions are:

- Can complaint-time features (text narrative, product/issue categories, channel, geography, and time features) predict monetary relief outcomes with useful accuracy?
- Which model family provides the best trade-off between precision, recall, and overall robustness for this imbalanced classification task?
- How should the decision threshold be adjusted to match operational priorities, such as capturing more potential monetary-relief cases versus reducing false alarms?
- Which complaint attributes appear most informative for distinguishing likely monetary-relief outcomes?

2.4 Target Variable

The target variable is a binary label derived from the complaint resolution outcome:

- 1: complaint ended with monetary relief.
- 0: complaint ended without monetary relief.

To preserve real-world validity, predictors are restricted to complaint-time information only. Fields that reveal or are strongly tied to post-resolution outcomes are excluded from the feature set used at inference time.

3 Data Preparation

3.1 Preparation Workflow Across Notebook 1 and Notebook 2

Data preparation was intentionally split into two stages:

- Notebook 1 (Data Loading + EDA): profile data quality, inspect missingness, and validate response-policy assumptions before modeling transformations.
- Notebook 2 (Preprocessing): implement the authoritative target-construction policy, clean and engineer features, enforce leakage boundaries, and export train/test-ready datasets.

This separation keeps exploratory diagnostics descriptive while keeping modeling inputs deterministic and reproducible.

3.2 Stage 1: EDA-Driven Preparation Decisions (Notebook 1)

Notebook 1 established the practical constraints that informed preprocessing:

- Sample-based workflow for memory safety and faster iteration on local hardware.
- High missingness in some fields (for example `Consumer complaint narrative` at about 73.8% missing in the working sample), indicating a need for robust text-presence handling.
- Strong class imbalance in the monetary-relief outcome proxy, indicating that downstream evaluation must go beyond accuracy.
- Clear evidence that some response statuses are unresolved or ambiguous and should not be used as supervised labels.

Notebook 1 also produced preparation-oriented artifacts in `reports/`, including:

- `eda_label_policy_summary.csv`
- `eda_missingness_by_proxy.csv`
- `eda_variable_preparation_summary.csv`

These artifacts were used as input checks before final preprocessing.

3.3 Stage 2: Authoritative Preprocessing (Notebook 2)

Notebook 2 converted EDA findings into the final supervised learning dataset.

3.3.1 Target Construction and Eligibility Filter

The target was created from `Company response to consumer` using a strict policy:

- Positive class (1): `Closed with monetary relief`.
- Negative class (0): clearly closed non-monetary outcomes (`Closed with explanation`, `Closed with non-monetary relief`, `Closed without relief`, and `Closed`).
- Excluded: unresolved or ambiguous outcomes (for example `In progress`, `Untimely response`, and `Closed with relief`).

From the sampled dataset, this policy yielded:

- 48,499 rows retained for supervised modeling.
- Monetary-relief positive rate of 1.3856%.

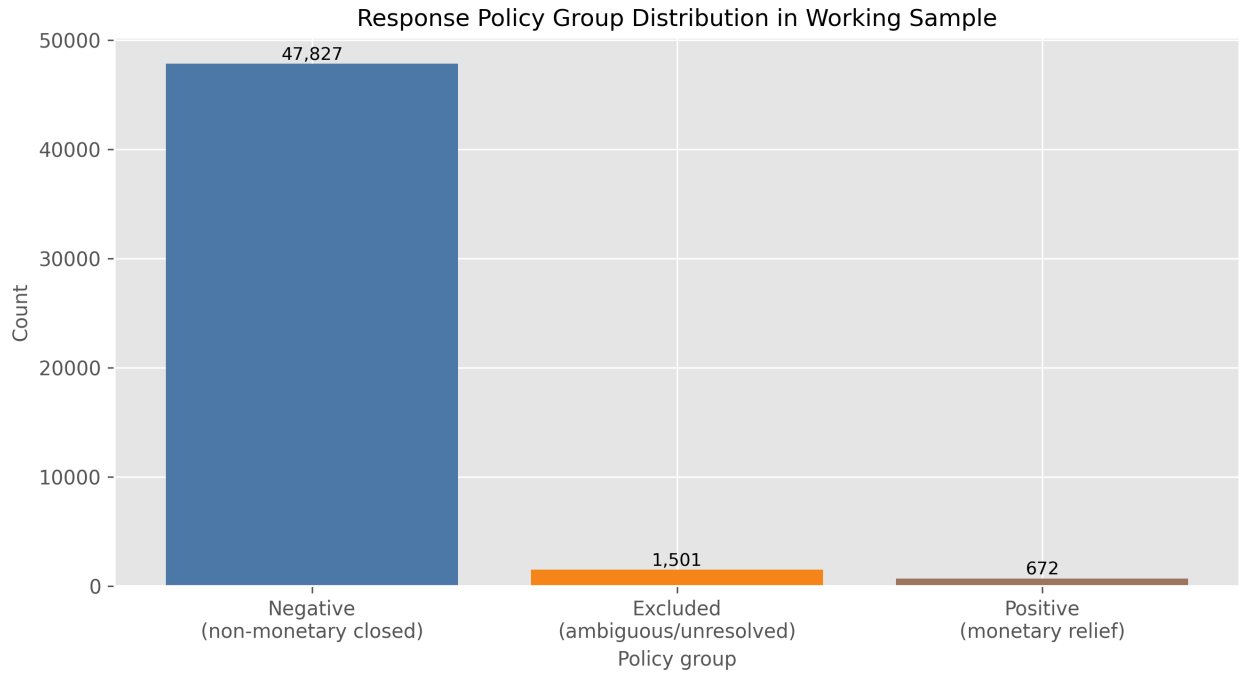


Figure 1: Distribution of response-policy groups in the working sample, showing why unresolved or ambiguous outcomes were excluded before supervised training.

This filter reduces label noise at the cost of sample size, which is appropriate for a high-stakes imbalanced classification task.

3.3.2 Text Cleaning and Narrative Features

Complaint narratives were transformed with a deterministic cleaning function:

- lowercase normalization,
- URL removal,
- email removal,
- cleanup of repeated redaction tokens,
- whitespace normalization.

From the cleaned text, two structured features were engineered:

- `narrative_length_raw` and capped `narrative_length` (99th percentile cap = 3138.22),
- `has_narrative` indicator (binary).

The percentile cap provides robust outlier control without deleting observations.

3.3.3 Temporal Feature Engineering

Date fields were parsed and transformed into complaint-time features:

- `year_received`,
- `month_received`,
- `quarter_received`,
- `days_to_send` (from `Date received` to `Date sent to company`).

Missing temporal derivatives were imputed with zero only at the numeric feature assembly stage to

maintain a complete matrix.

3.3.4 Categorical Encoding and Matrix Assembly

Selected categorical predictors were one-hot encoded:

- Product,
- Issue,
- State,
- Submitted via.

The final preprocessing output before model-specific text vectorization contained:

- 250 total features,
- 243 one-hot categorical columns,
- 6 numeric/engineered columns,
- 1 cleaned text field (`narrative_clean`) to be vectorized in Notebook 3.

3.3.5 Missing-Value Handling

Missing-value treatment followed feature semantics:

- Text: missing narratives converted to empty strings.
- Numeric engineered fields: filled with zeros where needed for matrix completeness.
- Encoded categorical indicators: naturally represented as zeros after one-hot encoding.

In the final assembled matrix, the tracked top features report 0% missingness in preprocessing outputs.

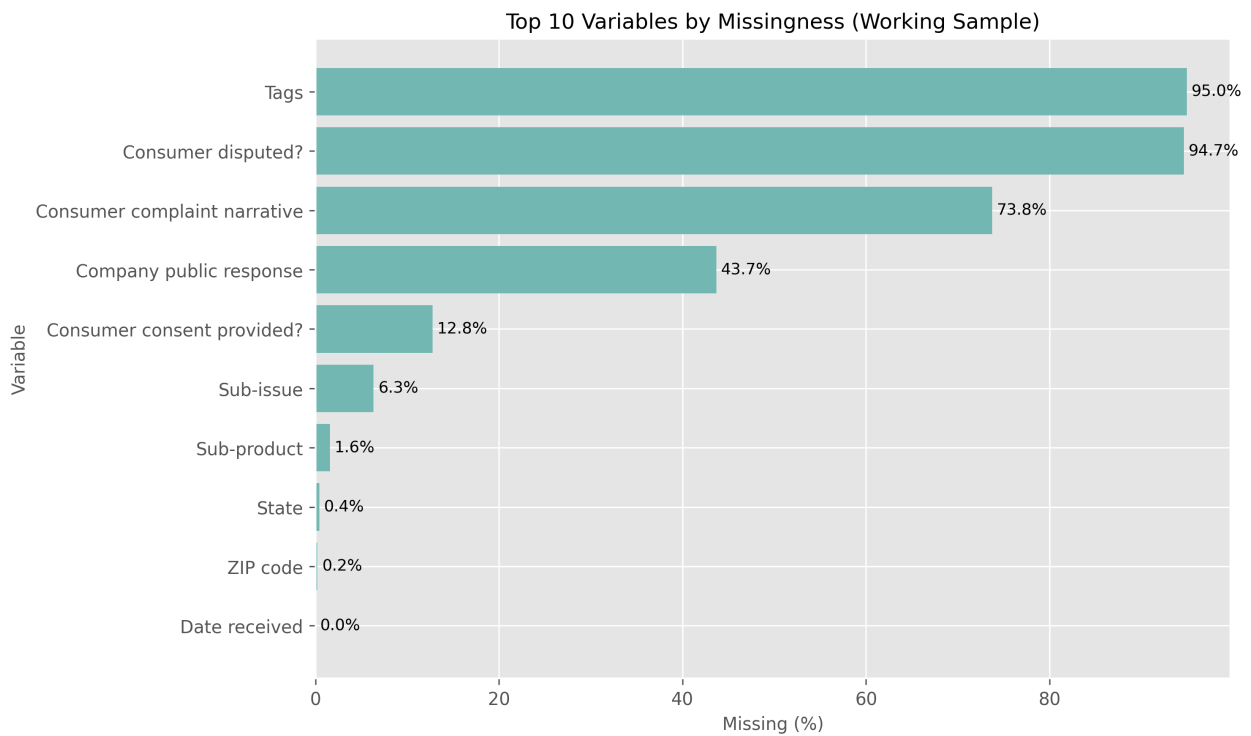


Figure 2: Top variables by missingness in the working sample. This motivated robust text-missing handling and selective feature inclusion.

3.3.6 Leakage Control

Leakage prevention was enforced as a hard rule:

- `Company response to consumer` was used only for target construction,
- target-related fields were excluded from predictors.

Notebook 2 validation reported `predictor_leakage_columns_found = 0`, confirming no direct leakage columns in `X`.

3.4 Exported Artifacts for Modeling

Notebook 2 exported all modeling inputs and documentation artifacts:

- `data/processed/train_features.csv`
- `data/processed/test_features.csv`
- `reports/preprocessing_summary.csv`
- `reports/feature_dictionary.csv`

The train/test split was stratified (80/20) to preserve class imbalance structure for fair downstream model comparison.

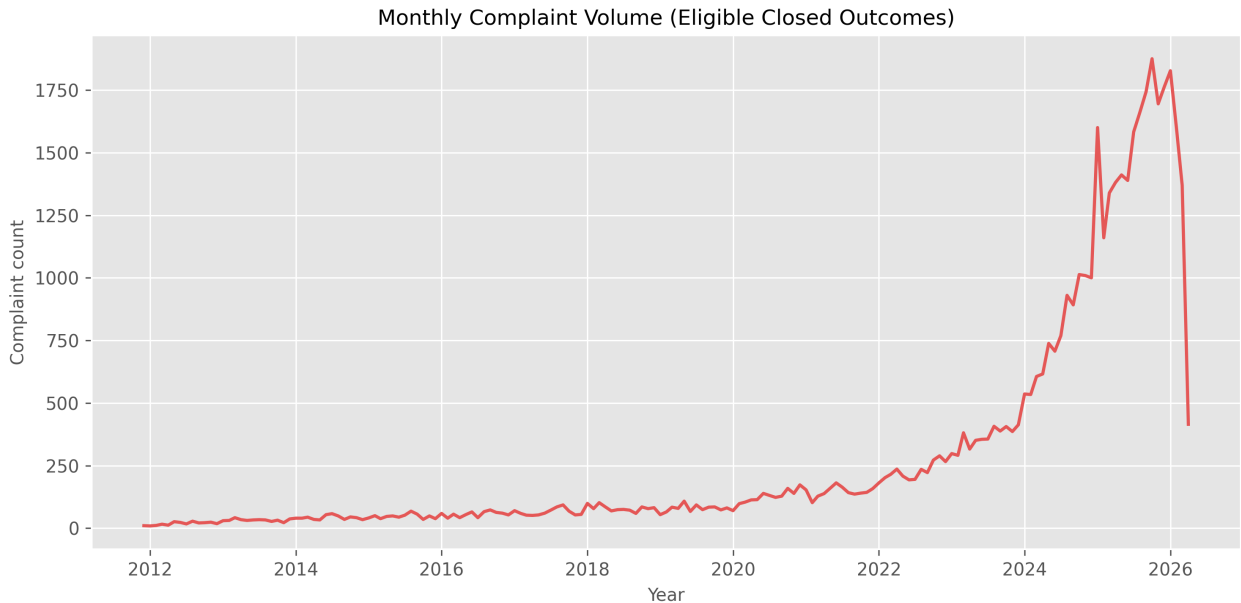


Figure 3: Monthly complaint volume trend for eligible closed outcomes, included to document temporal variation considered during preparation.

4 Methods and Analysis

4.1 Modeling Strategy

Modeling was performed in `03_modeling.ipynb` using the leakage-safe outputs from preprocessing (`train_features.csv` and `test_features.csv`). The split was stratified 80/20 to preserve the rare-event target distribution. Because the positive class rate is around 1.39%, the analysis emphasized imbalance-aware evaluation rather than raw accuracy.

The workflow was:

- load prepared train/test data,
- build text features with TF-IDF on `narrative_clean`,
- combine text and structured features when appropriate,
- train multiple model families under a shared evaluation protocol,
- compare both default-threshold and threshold-tuned performance,
- select a practical champion based on F1, recall-precision tradeoff, and confusion-matrix behavior.

4.2 Models Implemented and Rationale

The following models were implemented to cover complementary assumptions:

- Logistic Regression (TF-IDF text baseline): strong linear baseline for sparse high-dimensional text and easy to interpret.
- Naive Bayes (TF-IDF text baseline): classic probabilistic text classifier and fast benchmark.
- KNN with TruncatedSVD text reduction: non-parametric neighborhood baseline after dimensionality reduction.
- Random Forest (structured + text): captures non-linear interactions and mixed-feature effects.
- MLP Neural Network (structured + text): flexible non-linear learner with early stopping.
- Soft Voting Ensemble: combines probability outputs to test whether blending improves robustness.

This model set is appropriate because it spans linear, probabilistic, local, tree-based, neural, and ensemble families while remaining computationally feasible on the sampled dataset.

4.3 Evaluation Protocol

Each model was evaluated with:

- PR-AUC and ROC-AUC,
- Precision, Recall, F1,
- Balanced Accuracy and Cohen’s Kappa,
- confusion-matrix counts (TN, FP, FN, TP).

A threshold sweep (0.10 to 0.90) was then run for each model, and the best threshold per model was selected by F1. This is important in rare-event settings where a fixed 0.50 threshold often suppresses recall.

4.4 Methods Outputs

Notebook 3 generated reproducible artifacts used in this section:

- `reports/model_comparison_with_ensemble.csv`
- `reports/threshold_analysis.csv`
- `reports/model_comparison_detailed.csv`
- `reports/confusion_matrices.csv`
- `reports/top_model_curves.png`
- `reports/top_model_confusion_matrices.png`

5 Results and Interpretation

5.1 Default vs Tuned Performance

At the default threshold (0.50), most models produced near-zero recall because the task is highly imbalanced. After threshold tuning, performance improved substantially.

A concise view of the best tuned configurations is:

Model	Threshold	PR-AUC	F1	Precision	Recall	Kappa
Random Forest	0.10	0.2174	0.2993	0.1959	0.6343	0.2842
Voting Ensemble (5-model avg)	0.10	0.2310	0.2991	0.2177	0.4776	0.2855
Neural Network (ANN)	0.20	0.2223	0.2779	0.2000	0.4552	0.2638

Interpretation:

- The highest F1 comes from Random Forest at threshold 0.10.
- The highest PR-AUC comes from the Voting Ensemble, indicating strong ranking quality overall.
- The final model choice depends on operational preference: higher recall capture (Random Forest) versus tighter precision with fewer alerts (Voting).

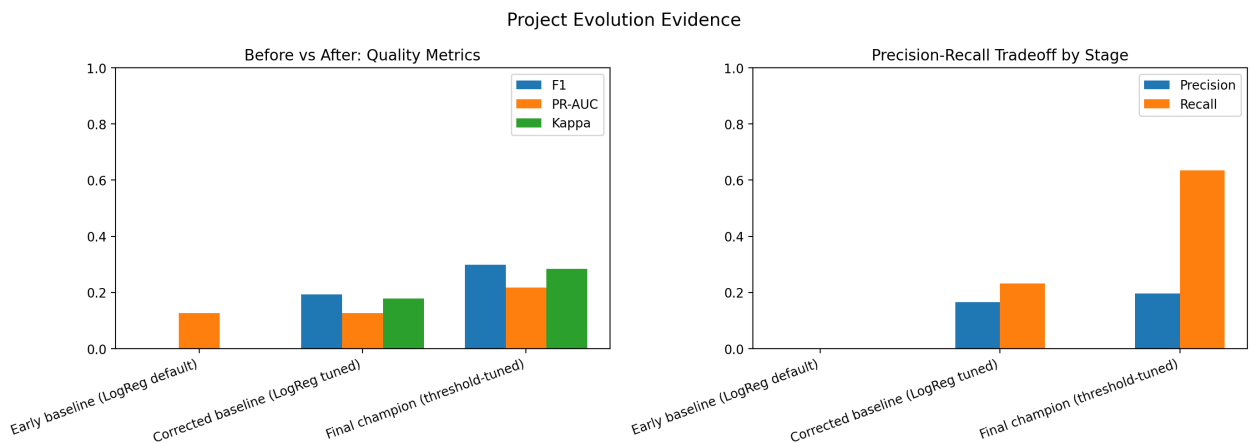


Figure 4: Project evolution evidence from baseline to tuned champion.

5.2 Curve-Level Interpretation

PR and ROC curves for top models show that all selected champions operate above no-skill baselines, but PR behavior is the most informative because positives are rare.

- Random Forest provides stronger recall at its tuned operating point.
- Voting Ensemble offers competitive F1 with a better precision profile.
- Neural Network remains viable but trails the top two on F1/Kappa.

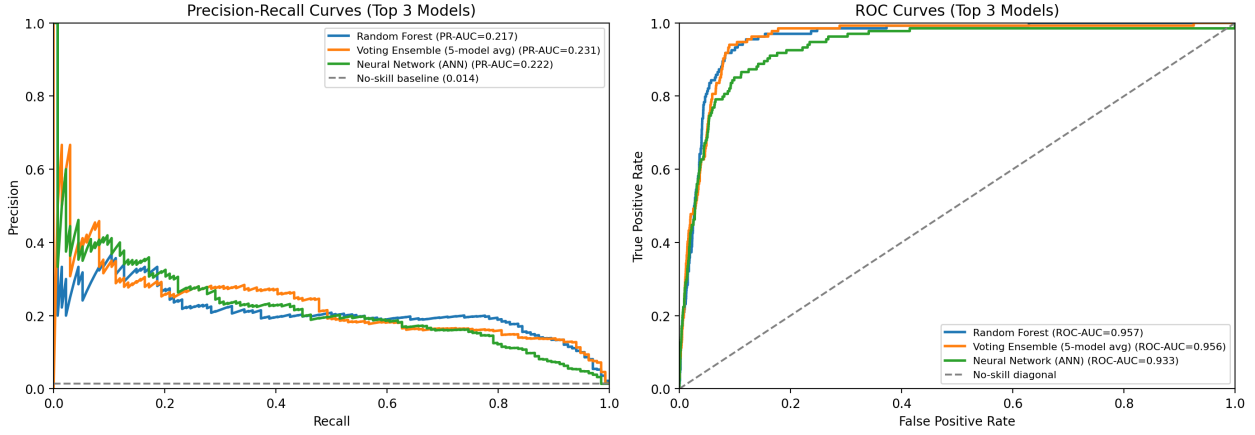


Figure 5: Top-model PR and ROC curves from threshold-tuned analysis.

5.3 Confusion-Matrix Insights

Using tuned thresholds:

- Random Forest: TP=85, FN=49, FP=349, TN=9217.
- Voting Ensemble: TP=64, FN=70, FP=230, TN=9336.

Business interpretation:

- Random Forest catches more true monetary-relief cases (higher recall) but triggers more false alerts.
- Voting triggers fewer alerts with better precision, but misses more true positive cases.

This is a classic triage tradeoff and should be aligned to operational cost preference (missed eligible cases vs review workload).

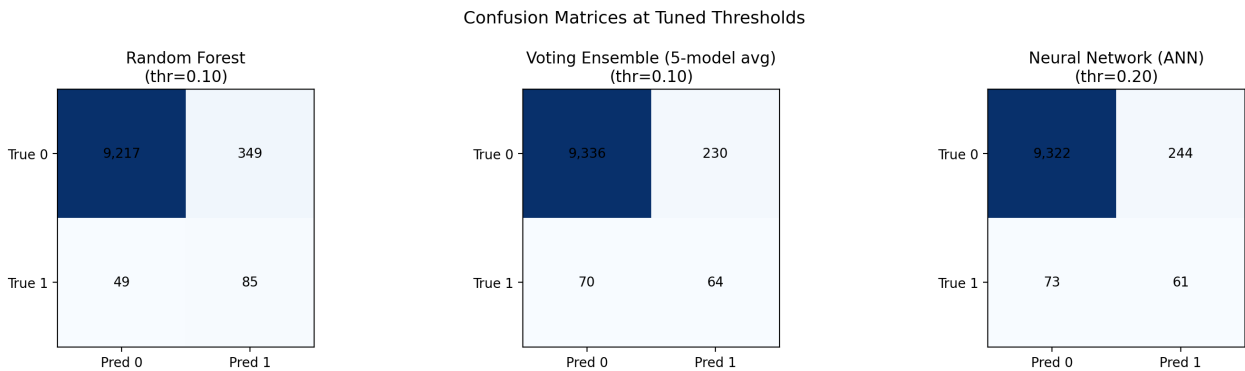


Figure 6: Confusion-matrix heatmaps for top tuned models.

5.4 Key Findings

Main findings from Notebook 3 are:

- Threshold tuning is essential; default 0.50 is not suitable for this rare-event task.

- Random Forest is the strongest recall-oriented champion (F1=0.2993, Recall=0.6343).
- Voting Ensemble is the strongest precision-oriented alternative (Precision=0.2177 with similar F1).
- Accuracy remains high for all models due to class imbalance and is not the primary decision metric.

Recommended operating policy for this project stage:

- Use Random Forest at threshold 0.10 when the objective is to maximize capture of potential monetary-relief cases.
- Use Voting Ensemble at threshold 0.10 when review capacity is constrained and precision is prioritized.

6 Conclusion

6.1 Objective Recap

The project objective was to predict whether a CFPB complaint will end with monetary relief using only complaint-time information. We focused on a leakage-safe binary classification setup and prioritized practical triage usefulness under severe class imbalance, rather than headline accuracy.

6.2 Methods Recap

We prepared a supervised dataset with strict target eligibility rules, engineered text/temporal/structured features, and compared six model configurations in Notebook 3: Logistic Regression, Naive Bayes, KNN (SVD-reduced text), Random Forest, MLP Neural Network, and a soft-voting ensemble. Evaluation used PR-AUC, ROC-AUC, Precision, Recall, F1, Kappa, and confusion matrices. We then performed threshold tuning (0.10 to 0.90) to choose operationally useful settings.

6.3 Main Results

At the default threshold (0.50), minority-class detection was weak across most models. After threshold tuning, the strongest recall-oriented configuration was Random Forest at threshold 0.10 (F1=0.2993, Recall=0.6343, Precision=0.1959, Kappa=0.2842). The strongest precision-oriented alternative was Voting Ensemble at threshold 0.10 (F1=0.2991, Precision=0.2177, Recall=0.4776, Kappa=0.2855), with the highest PR-AUC overall (0.2310). These results confirm that threshold choice is as important as model family in this rare-event context.

6.4 Business Insights

For complaint triage, the recommended operating policy depends on review capacity and risk tolerance. If the primary goal is to capture as many potential monetary-relief cases as possible, Random Forest at 0.10 is preferred. If review capacity is constrained and false alerts must be reduced, Voting Ensemble at 0.10 is a strong alternative. In both cases, periodic threshold recalibration and ongoing monitoring of precision-recall tradeoffs are necessary as complaint patterns evolve.

6.5 AI Tool Disclosure

GitHub Copilot was used as an agentic coding assistant to help draft, revise, and organize parts of the report and notebook code. All AI-assisted outputs were reviewed and edited by the project authors before inclusion, and the final analysis and conclusions reflect human oversight and approval.